

A
Project Report
On
Argex
Submitted by
Uplaksh Tyagi
Roll No.: 25368
MCA–I
SEM–II
Under the guidance of
Prof. Monalisa Bhinge
For the Academic Year 2025-26



Sinhgad Institutes

Sinhgad Technical Education Society's
Sinhgad Institute of Management

Vadgaon Bk Pune 411041

(Affiliated to SPPU Pune & Approved by AICTE New Delhi)

Prof. M. N. Navale
M.E. (ELECT.), MIE, MBA
FOUNDER PRESIDENT

Dr. (Mrs.) Sunanda M. Navale
B.A., MPM, Ph.D
FOUNDER SECRETARY

Dr. Chandrani Singh
MCA, ME, (Com. Sci.), Ph.D
DIRECTOR - MCA

Date:

CERTIFICATE

This is to certify that Mr. Uplaksh Tyagi has successfully completed his project work entitled “**Argex**” in partial fulfillment of MCA – I SEM –II Mini Project for the year 2025-2026. He has worked under our guidance and direction.

Prof. Monalisa Bhinge
Project Guide

Dr. Chandrani Singh
Director, SIOM-MCA

Date:

Place: Pune

Celebrating 25 Years
OF ACADEMIC EXCELLENCE

DECLARATION

I certify that the work contained in this report is original and has been done by me under the guidance of my guide.

- The work has not been submitted to any other Institute for any degree or diploma.
- I have followed the guidelines provided by the Institute in preparing the report.
- I have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Institute.
- Whenever I have used materials (data, theoretical analysis, figures, and text) from _____ other sources, I have given due credit to them by citing them in the text of the report and giving their details in the references.

Name and Signature of Project Team Members:

Sr. No.	Roll No.	Name of students	Signature of students
1	25368	Uplaksh Tyagi	

ACKNOWLEDGEMENT

It is very difficult task to acknowledge all those who have been of tremendous help in this project. I would like to thank my respected guide **Prof. Monalisa Bhinge** for providing me necessary facilities to complete my project and for their guidance and encouragement in completing my project successfully without which it wouldn't be possible. I wish to convey my special thanks and immeasurable feelings of gratitude towards **Dr. Chandrani Singh, Director, SIOM-MCA**. I wish to convey my special thanks to all teaching and non-teaching staff members of **Sinhgad Institute of Management, Pune** for their support.

Thank You

Yours Sincerely,

Uplaksh Tyagi

INDEX

Sr. No.	Chapter	Page No.
1	CHAPTER 1: INTRODUCTION	6
1.1	Abstract	6
1.2	Existing System and Need for System	7
1.3	Scope of System	8
1.4	Operating Environment Hardware and Software	9
1.5	Brief Description of Technology Used	10
2	CHAPTER 2: PROPOSED SYSTEM	11
2.1	Feasibility Study	11
2.2	Objectives of the proposed system	12
2.3	Users of the system	12
3	CHAPTER 3: ANALYSIS AND DESIGN	13
3.1	Entity Relationship Diagram (ERD)	14
3.2	Class Diagram	15
3.3	Use Case Diagrams	16
3.4	Activity Diagram	21
3.5	Sequence Diagram	27
3.6	Component Diagram	31
3.7	Module and Hierarchy Diagram	32
3.8	Table Design	33
3.9	Data Dictionary	40
3.10	Sample Input and Output Screens (Screens must have valid data. All reports must have at-least 5 valid records.)	46
4	CHAPTER 4: CODING Sample code	53
5	CHAPTER 5: LIMITATIONS OF SYSTEM	61
6	CHAPTER 6: PROPOSED ENHANCEMENTS	62
7	CHAPTER 7: CONCLUSION	62
8	CHAPTER 8: BIBLIOGRAPHY	63

Chapter 1: INTRODUCTION

1.1 Abstract

ARGEX is a secure real-time online auction platform designed to provide a competitive and interactive bidding environment integrated with advanced cybersecurity monitoring features. Traditional auction systems primarily focus on transaction handling and bidding operations but often lack mechanisms for behavioral monitoring, anomaly detection, and real-time security analysis. ARGEX addresses these limitations by combining a modern auction ecosystem with integrated security intelligence capabilities.

The system allows users to register, authenticate, participate in auctions, manage watchlists, and place bids in real time through a responsive web-based interface. Sellers can create and manage auctions, while bidders can participate in competitive bidding using manual as well as automated bidding mechanisms. The platform also supports role-based access control with dedicated functionalities for bidders, sellers, defenders, and administrators.

One of the key highlights of ARGEX is its integrated cybersecurity monitoring architecture. The platform continuously analyzes bidding behavior and transaction activity to detect suspicious actions such as rapid bidding, abnormal bid increments, and unusual marketplace activity. Detected incidents are processed through a risk-scoring engine and displayed within a dedicated Defender Dashboard, enabling monitoring and response workflows similar to lightweight Security Operations Center (SOC) environments.

The project uses FastAPI as the backend framework and React with Tailwind CSS for the frontend interface. PostgreSQL is used as the primary database for handling transactional and security-related data. Real-time communication is achieved through WebSocket integration, enabling live bid updates and dynamic auction room interactions. Additional features such as wallet balance management, currency conversion, email verification, PIN-based secure authentication, auto-bidding, notification systems, and responsive dark-mode UI further enhance the overall usability and realism of the platform.

ARGEX demonstrates the integration of full-stack web development, real-time systems, role-based security architecture, and cyber-aware transaction monitoring within a single unified platform. The project aims to provide a scalable and intelligent auction environment that not only supports marketplace functionality but also emphasizes operational security, monitoring, and system integrity.

1.2 Existing System and Need for System

- **Existing System**

Existing online auction systems mainly focus on basic functionalities such as user registration, auction creation, and bid placement. Most traditional platforms provide limited real-time interaction and lack advanced security mechanisms to monitor suspicious bidding behavior or fraudulent activities. Many systems also suffer from delayed updates, weak role management, poor responsiveness, and insufficient monitoring features, which reduce overall platform reliability and user trust.

- **Need for System**

The need for ARGEX arose from the requirement for a secure and intelligent real-time auction platform capable of handling both marketplace operations and security monitoring simultaneously. ARGEX aims to provide live bidding, auto-bidding, responsive user interaction, role-based access control, and integrated cybersecurity features such as anomaly detection, risk scoring, and incident monitoring. The system improves transparency, operational security, and user experience while creating a more reliable and modern online auction environment.

1.3 Scope of System

- **Real-Time Auction Management:**
The system supports creation, management, and monitoring of real-time online auctions with live bid updates.
- **User Role Management:**
ARGEX provides role-based access control for bidders, sellers, defenders, and administrators with different functionalities for each role.
- **Secure Bidding System:**
Users can place manual as well as automated bids with validation and balance checking mechanisms.
- **Cybersecurity Monitoring:**
The platform includes anomaly detection, risk scoring, and incident monitoring to identify suspicious auction activities.
- **Live Communication:**
WebSocket integration enables real-time bid synchronization and live marketplace interaction.
- **Wallet and Currency Management:**
Users can manage account balances and view auction values in multiple currencies using real-time conversion.
- **Notification and Alert System:**
The system provides real-time notifications for bidding activity, seller approvals, and security incidents.
- **Responsive User Interface:**
ARGEX supports modern responsive design with dark mode compatibility for multiple device types.

1.4 Operating Environment Hardware and Software:

Server-side requirement

Software Requirement	Hardware Requirement
Operating System: Windows 10 or above/Linux	Processor: Intel Core i5 or above
Front End: React.js, Tailwind CSS, Vite	RAM: 8GB or above
Back End: FastAPI (Python)	HDD/SSD: 512 GB or above
Database: PostgreSQL	Stable Internet Connection
Additional tools: WebSocket, JWT Authentication, SMTP Services	

Client-side requirement

Software Requirement	Hardware Requirement
Operating System: Windows 10 or above/ Linux/ macOS	Processor: Intel Core i3 or above
Web Browser: Google Chrome, Mozilla Firefox, Microsoft Edge	RAM: 4GB or above
Internet Connection Required	HDD/SSD: 256 GB or above

1.5 Brief Description of Technology Used

- **React.js:**
React.js is used for developing the frontend user interface of ARGEX. It provides a component-based architecture for building responsive and interactive web pages.
- **Tailwind CSS:**
Tailwind CSS is used for designing the user interface with modern responsive layouts, dark mode support, and customizable styling.
- **FastAPI:**
FastAPI is used as the backend framework for handling APIs, authentication, auction management, bidding operations, and security functionalities.
- **PostgreSQL:**
PostgreSQL is used as the relational database management system for storing user data, auctions, bids, notifications, incidents, and transaction records.
- **WebSocket:**
WebSocket technology is used to implement real-time bid updates and live auction synchronization between users.
- **JWT Authentication:**
JSON Web Tokens (JWT) are used for secure user authentication and role-based access control.
- **SMTP Email Service:**
SMTP services are used for email verification, OTP delivery, and notification-related functionalities.
- **Python:**
Python is used as the primary programming language for implementing backend services, business logic, and security monitoring modules.

Chapter 2: PROPOSED SYSTEM

2.1 Feasibility Study

- **Technical Feasibility:**

The development of ARGEX is technically feasible as it uses modern and widely supported technologies such as React.js, FastAPI, PostgreSQL, and WebSocket communication. The required development tools, frameworks, and libraries are easily available and compatible with standard computing environments. The system architecture supports real-time communication, secure authentication, role-based access control, and cybersecurity monitoring functionalities effectively.

- **Operational Feasibility:**

ARGEX is operationally feasible because the system is designed with a user-friendly interface and role-based workflows for bidders, sellers, defenders, and administrators. The platform automates auction handling, bid monitoring, notification management, and security incident tracking, reducing manual effort and improving operational efficiency.

- **Economic Feasibility:**

The project is economically feasible because it uses open-source technologies and development tools, minimizing implementation cost. The software stack including React.js, FastAPI, PostgreSQL, and Python can be deployed without expensive licensing requirements, making the system cost-effective for development and maintenance.

- **Schedule Feasibility:**

The project is schedule feasible as the system was developed in modular phases including authentication, auction management, bidding engine, security monitoring, real-time communication, and frontend integration. This phased development approach allowed systematic implementation, testing, and refinement within the planned project timeline.

2.2 Objectives of the Proposed System

- **Develop a Real-Time Auction Platform:**
To create a responsive online auction system that supports live bidding and real-time auction updates.
- **Implement Secure Authentication:**
To provide secure user authentication using email verification, PIN-based login, and role-based access control mechanisms.
- **Enable Smart Bidding Features:**
To support manual bidding, automated bidding, bid validation, and wallet-based transaction handling.
- **Integrate Cybersecurity Monitoring:**
To detect suspicious activities such as rapid bidding, abnormal bid patterns, and unauthorized behavior using risk analysis and incident monitoring.
- **Provide Role-Based Functionalities:**
To implement dedicated functionalities for bidders, sellers, defenders, and administrators within a single platform.
- **Improve User Experience:**
To provide a modern, responsive, and interactive user interface with real-time notifications and dark mode support.
- **Ensure Scalability and Maintainability:**
To develop the system using modular architecture and modern technologies for easier scalability and future enhancements.

2.3 Users of the System

- **Bidder:**
Bidders participate in auctions by placing manual or automated bids, managing watchlists, and monitoring live auction activity.
- **Seller:**
Sellers create and manage auctions, monitor bidding activity on their products, and receive auction earnings through the wallet system.
- **Defender:**
Defenders monitor suspicious activities, analyze risk scores, view security incidents, and maintain platform security through the Defender Dashboard.
- **Administrator:**
Administrators manage users, approve seller requests, manage auction categories, and maintain overall platform operations and access control.

CHAPTER 3: ANALYSIS AND DESIGN

This chapter describes the overall system analysis and design of ARGEX. It includes the architectural structure, workflow diagrams, UML diagrams, database design, and data organization used in the system. Various diagrams such as use case diagrams, activity diagrams, sequence diagrams, class diagrams, and entity relationship diagrams are used to represent the interaction between different modules and users within the platform. The chapter also presents detailed table designs and data dictionary representations that define the database structure and relationships between different entities. These design components provide a clear understanding of the system functionality, data flow, and implementation architecture of ARGEX.

Entity Relationship Diagram (ERD)

Class Diagram

Use Case 1

Use Case 2

Use Case 3

\

Use Case 4

Use Case 5

Activity 1

Activity 2

Activity 3

Activity 4

Activity 5

Activity 6

Sequence 1

Sequence 2

Sequence 3

Sequence 4

Component Diagram

Module and Hierarchy Diagram

3.8 Table Design

3.8.1 users		
Field name	Data type	Description
id	Integer (Primary key)	Unique identifier for each user
username	Varchar	Stores username of the user
email	Varchar	Store email address of the user
password_hash	Varchar	Stores encrypted password of the user
role	User-defined	Defines role of the user
cumulative_risk_score	Double Precision	Stores overall calculated risk score of the user
is_suspected	Boolean	Indicates whether the user is marked a suspicious
is_blocked	Boolean	Indicates whether the user is blocked
account_balance	Double Precision	Stores wallet balance of the user
preferred_currency	Varchar	Stores preferred currency selected by the user
phone_number	Varchar	Stores contact phone number of the user
is_verified	Boolean	Indicates whether the user account is verified
security_pin_hash	Varchar	Stores encrypted security PIN of the user
pin_failed_attempts	Integer	Stores count of failed PIN verification
pin_locked_until	Timestamp with TimeZone	Stores temporary PIN lock expiration time

3.8.2 auctions

Field Name	Data Type	Description
id	Integer (Primary key)	Unique identifier for each auction
title	Varchar	Stores title of the auction item
description	Varchar	Stores detailed description of the auction item
starting_price	Double Precision	Stores initial starting price of the auciton
current_price	Double Precision	Stores current highest bid amount of the auction
seller_id	Integer	Stores reference ID of the seller who created the auction
start_time	Timestamp with TimeZone	Stores scheduled starting time of the auciton
end_time	Timestamp with TimeZone	Stores ending time of the auction
status	User-defined	Defines current status of the auction
category_id	Integer	Stores reference ID of the auction category
extension_count	Integer	Stores number of auction time extensions applied
auction_currency	Varchar	Stores currency selected for the auction transactions

3.8.3 audit_logs

Field Name	Data Type	Description
id	Integer (Primary key)	Unique identifier for each audit log entry
user_id	Integere	Stores reference ID of the user performing the auction
action_type	Varchar	Stores type of action performed in the system
entity_type	Varchar	Stores type of entity affected
entity_id	Integer	Stores reference Id of the affected entity
details	JSON	Stores additional structured information related to the auction
timestamp	Timestamp with TimeZone	Stores data and time when the action was recorded

3.8.4 auto_bids

Field Name	Data Type	Description
id	Integer (Primary key)	Unique identifier for each auto-bid entry
user_id	Integer	Stores reference ID of the user who enabled auto-bidding
auction_id	Integer	Stores reference ID of the associated auction
max_bid	Double Precision	Stores maximum bid limit set by the user
current_bid	Double Precision	Stores current automatically placed bid amount
is_active	Boolean	Indicates whether auto-bidding is currently active

3.8.5 bids

Field Name	Data Type	Description
id	Integer (Primary key)	Unique identifier for each bid entry
amount	Double Precision	Stores bid amount placed by the bidder
bidder_id	Integer	Stores reference ID of the bidder placing the bid
auction_id	Integer	Stores reference ID of the associated auction
created_at	Timestamp with TimeZone	Stores date and time when the bid was placed
is_auto	Boolean	Indicates whether the bid was placed automatically through auto-bidding

3.8.6 categories

Field Name	Data Type	Description
id	Integer (Primary key)	Unique identifier for each category
name	Varchar	Stores name of the auction category

3.8.7 defender_action_logs

Field Name	Data Type	Description
id	Integer (Primary key)	Unique identifier for each defender action logs
defender_id	Integer	Stores reference ID of the defender performing the action
action_type	User-Defined	Stores type of security action performed by the defender
target_user_id	Integer	Stores reference ID of the targeted user involved in the action
incident_id	Integer	Stores reference ID of the related security incident
timestamp	Timestamp with TimeZone	Stores data and time when the defender action was recorded

3.8.8 notifications

Field Name	Data Type	Description
id	Integer (Primary key)	Unique identifier for each notification
user_id	Integer	Stores reference ID of the user receiving the notification
type	Varchar	Stores type of notification generated by the system
message	Varchar	Stores notification message content
is_read	Boolean	Indicates whether the notification has been read by the user
created_at	Timestamp with TimeZone	Stores data and time when the notification was generated

3.8.9 security_incidents

Field Name	Data Type	Description
id	Integer (Primary key)	Unique identifier for each security incident
user_id	Integer	Stores reference ID of the user associated with the incident
auction_id	Integer	Stores reference ID of the related auction
incident_type	Varchar	Stores type of suspicious activity detected
severity	User-defined	Defines severity level of the security incident
risk_score	Double Precision	Stores calculated risk score of the incident
description	Text	Stores detailed description of the detected incident
created_at	Timestamp with TimeZone	Stores date and time when the incident was generated
status	User-defined	Defines current status of the incident

3.8.10 seller_requests

Field Name	Data Type	Description
id	Integer (Primary key)	Unique identifier for each seller request
user_id	Integer	Stores reference ID of the user requesting seller access
status	User-defined	Defines current approval status of the seller request
created_at	Timestamp with TimeZone	Stores date and time when the seller request was created

3.8.11 watchlists

Field Name	Data type	Description
id	Integer (Primary key)	Unique identifier for each watchlist entry
user_id	Integer	Stores reference ID of the user managing the watchlist
auction_id	Integer	Stores reference ID of the auction added to the watchlist

3.9 Data Dictionary

Table Name	Field name	Data type	Description
users	id	Integer (Primary key)	Unique identifier for each user
	username	Varchar	Stores username of the user
	email	Varchar	Store email address of the user
	password_hash	Varchar	Stores encrypted password of the user
	role	User-defined	Defines role of the user
	cumulative_risk_score	Double Precision	Stores overall calculated risk score of the user
	is_suspected	Boolean	Indicates whether the user is marked a suspicious
	is_blocked	Boolean	Indicates whether the user is blocked
	account_balance	Double Precision	Stores wallet balance of the user
	preferred_currency	Varchar	Stores preferred currency selected by the user
	phone_number	Varchar	Stores contact phone number of the user
	is_verified	Boolean	Indicates whether the user account is verified
	security_pin_hash	Varchar	Stores encrypted security PIN of the user
	pin_failed_attempts	Integer	Stores count of failed PIN verification

	pin_locked_until	Timestamp with TimeZone	Stores temporary PIN lock expiration time
auctions	id	Integer (Primary key)	Unique identifier for each auction
	title	Varchar	Stores title of the auction item
	description	Varchar	Stores detailed description of the auction item
	starting_price	Double Precision	Stores initial starting price of the auciton
	current_price	Double Precision	Stores current highest bid amount of the auction
	seller_id	Integer	Stores reference ID of the seller who created the auction
	start_time	Timestamp with TimeZone	Stores scheduled starting time of the auciton
	end_time	Timestamp with TimeZone	Stores ending time of the auction
	status	User-defined	Defines current status of the auction
	category_id	Integer	Stores reference ID of the auction category
	extension_count	Integer	Stores number of auction time extensions applied
	auction_currency	Varchar	Stores currency selected for the auction transactions
audit_logs	id	Integer (Primary key)	Unique identifier for each audit log entry
	user_id	Integere	Stores reference ID of the user

			performing the auction
	action_type	Varchar	Stores type of action performed in the system
	entity_type	Varchar	Stores type of entity affected
	entity_id	Integer	Stores reference Id of the affected entity
	details	JSON	Stores additional structured information related to the auction
	timestamp	Timestamp with TimeZone	Stores data and time when the action was recorded
auto_bids	id	Integer (Primary key)	Unique identifier for each auto-bid entry
	user_id	Integer	Stores reference ID of the user who enabled auto-bidding
	auction_id	Integer	Stores reference ID of the associated auction
	max_bid	Double Precision	Stores maximum bid limit set by the user
	current_bid	Double Precision	Stores current automatically placed bid amount
	is_active	Boolean	Indicates whether auto-bidding is currently active
bids	id	Integer (Primary key)	Unique identifier for each bid entry
	amount	Double Precision	Stores bid amount placed by the bidder
	bidder_id	Integer	Stores reference ID

			of the bidder placing the bid
	auction_id	Integer	Stores reference ID of the associated auction
	created_at	Timestamp with TimeZone	Stores date and time when the bid was placed
	is_auto	Boolean	Indicates whether the bid was placed automatically through auto-bidding
categories	id	Integer (Primary key)	Unique identifier for each category
	name	Varchar	Stores name of the auction category
defender_action_logs	id	Integer (Primary key)	Unique identifier for each defender action logs
	defender_id	Integer	Stores reference ID of the defender performing the action
	action_type	User-Defined	Stores type of security action performed by the defender
	target_user_id	Integer	Stores reference ID of the targeted user involved in the action
	incident_id	Integer	Stores reference ID of the related security incident
	timestamp	Timestamp with TimeZone	Stores data and time when the defender action was recorded
notifications	id	Integer (Primary	Unique identifier for

		key)	each notification
	user_id	Integer	Stores reference ID of the user receiving the notification
	type	Varchar	Stores type of notification generated by the system
	message	Varchar	Stores notification message content
	is_read	Boolean	Indicates whether the notification has been read by the user
	created_at	Timestamp with TimeZone	Stores data and time when the notification was generated
security_incidents	id	Integer (Primary key)	Unique identifier for each security incident
	user_id	Integer	Stores reference ID of the user associated with the incident
	auction_id	Integer	Stores reference ID of the related auction
	incident_type	Varchar	Stores type of suspicious activity detected
	severity	User-defined	Defines severity level of the security incident
	risk_score	Double Precision	Stores calculated risk score of the incident
	description	Text	Stores detailed description of the detected incident

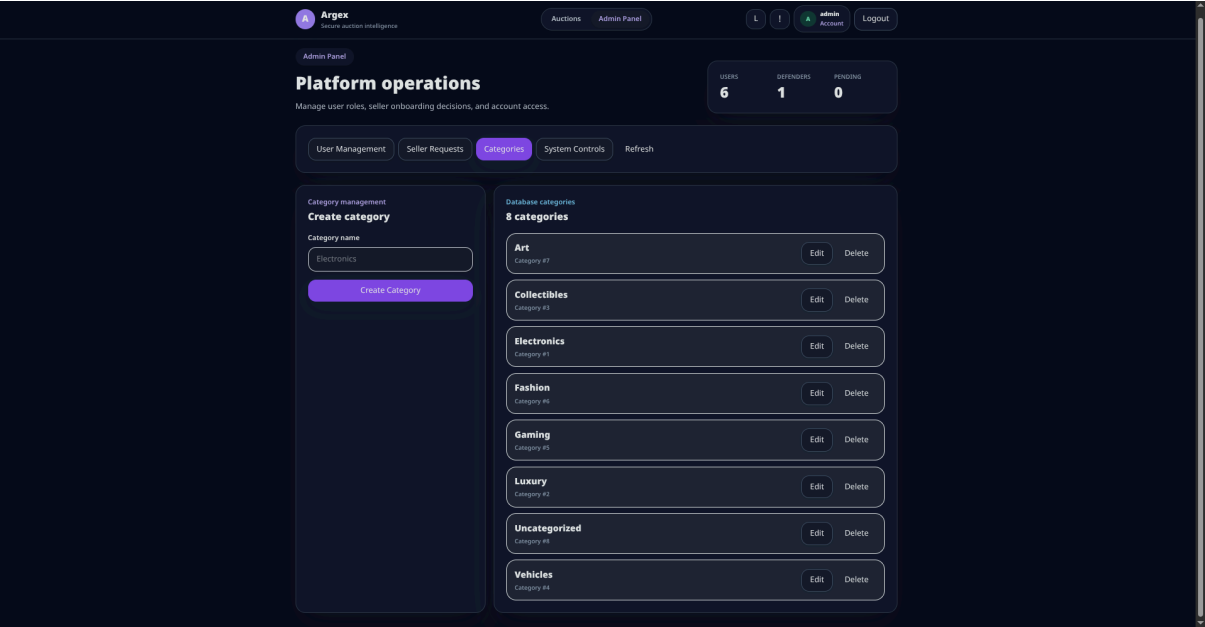
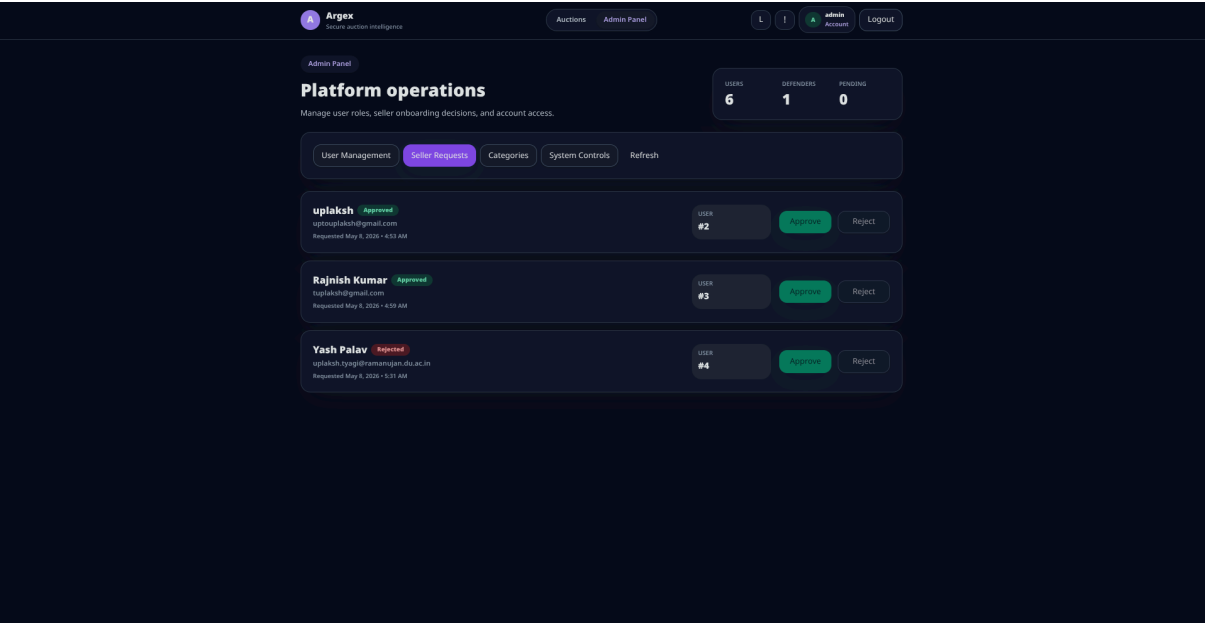
	created_at	Timestamp with TimeZone	Stores data and time when the incident was generated
	status	User-defined	Defines current status of the incident
seller_requests	id	Integer (Primary key)	Unique identifier for each seller request
	user_id	Integer	Stores reference ID of the user requesting seller access
	status	User-defined	Defines current approval status of the seller request
	created_at	Timestamp with TimeZone	Stores date and time when the seller request was created
watchlists	id	Integer (Primary key)	Unique identifier for each watchlist entry
	user_id	Integer	Stores reference ID of the user managing the watchlist
	auction_id	Integer	Stores reference ID of the auction added to the watchlist

3.10 Sample Input and Output Screens

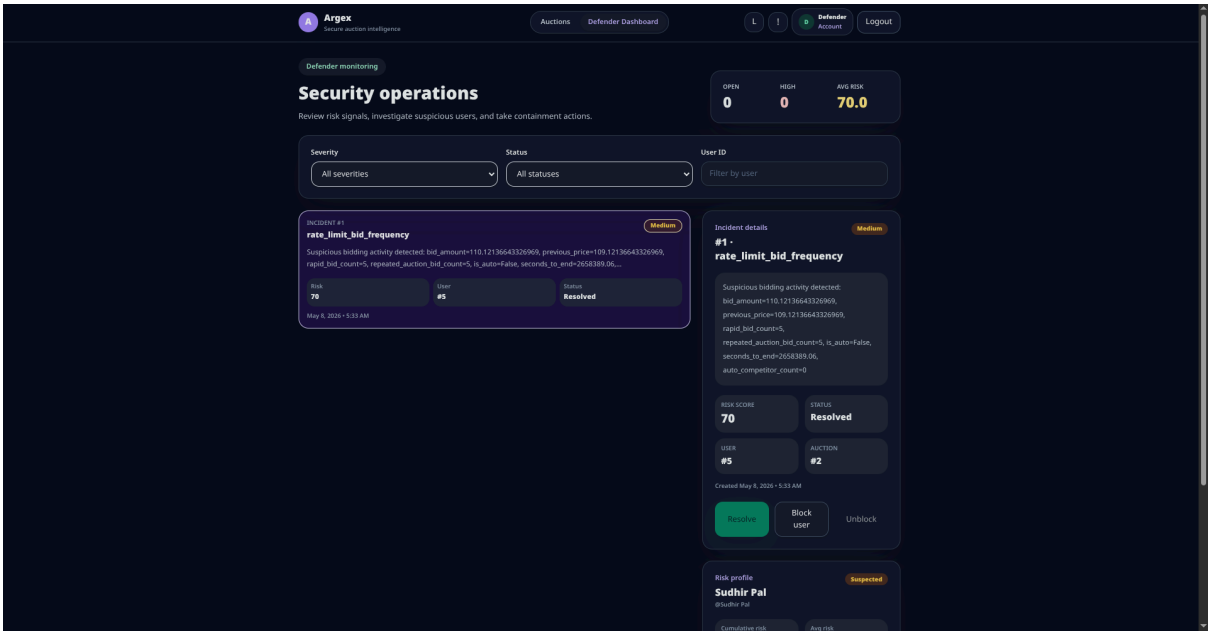
admin dashboards

The screenshot displays the Argex Admin Panel interface. At the top, the header includes the Argex logo, navigation tabs for 'Auctions' and 'Admin Panel', and user account information for 'admin'. The main section is titled 'Platform operations' with a subtitle 'Manage user roles, seller onboarding decisions, and account access.' To the right of this title is a summary card showing 'USERS: 6', 'DEFENDERS: 1', and 'PENDING: 0'. Below the title, there are tabs for 'User Management', 'Seller Requests', 'Categories', 'System Controls', and a 'Refresh' button. The 'User Management' tab is active, showing a list of users. Each user entry includes their name, email, account ID, status (Active), role (Admin, Seller, Bidder, Defender), and an 'Assign defender' button.

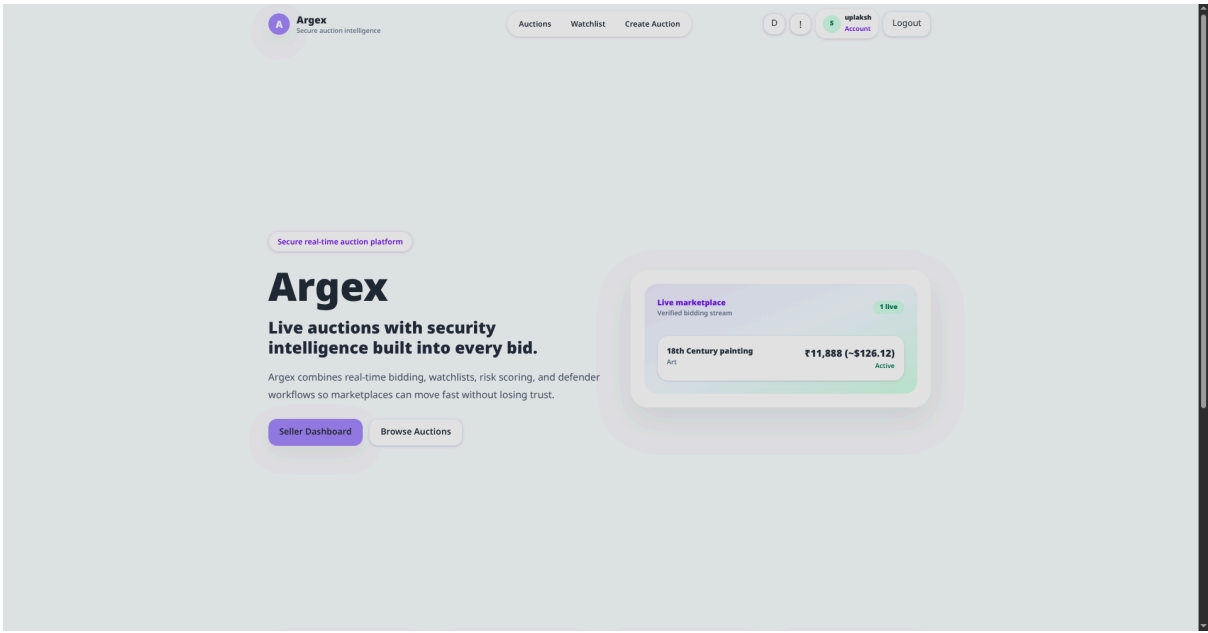
NAME	EMAIL	ACCOUNT ID	STATUS	ROLE	ACTION
admin	uplakshyagi1234@gmail.com	Account ID 1	Active	Admin	Assign defender
uplaksh	uplakshyagi1234@gmail.com	Account ID 2	Active	Seller	Assign defender
Rajnish Kumar	tuplaksh@gmail.com	Account ID 3	Active	Seller	Assign defender
Yash Palav	uplaksh.yagi1234@gmail.com	Account ID 4	Active	Bidder	Assign defender
Sudhir Pal	uplakshyagi1234@gmail.com	Account ID 5	Active	Bidder	Assign defender
Defender	uplaksh.audf@gmail.com	Account ID 6	Active	Defender	Assign defender

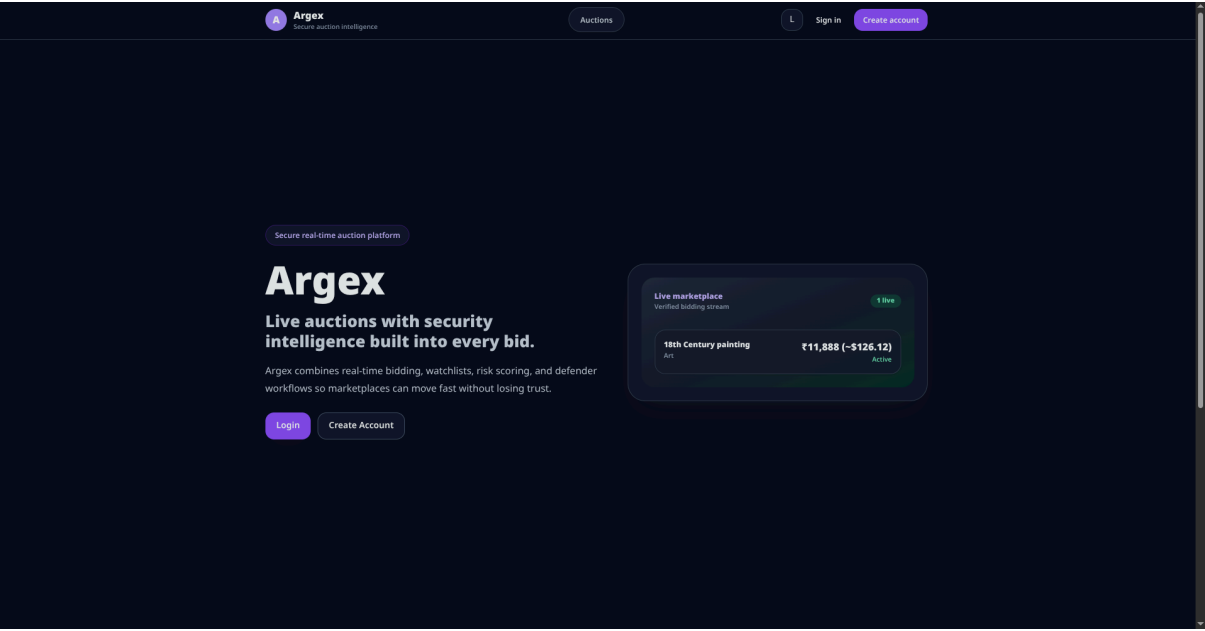


defender dashboard

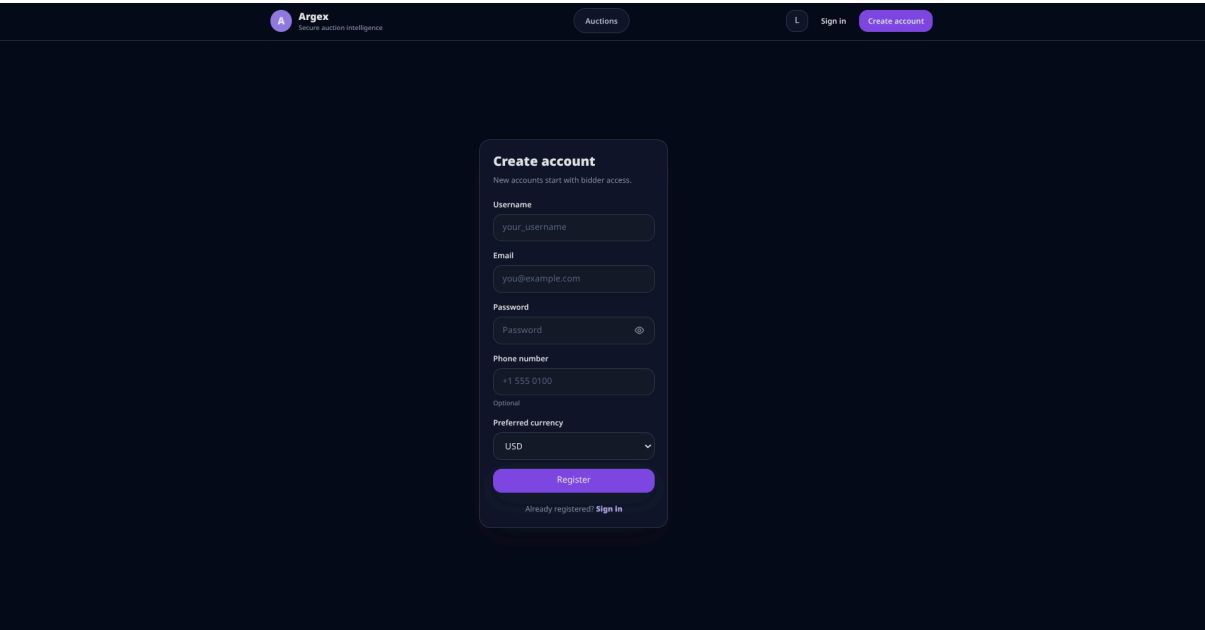


landing pages

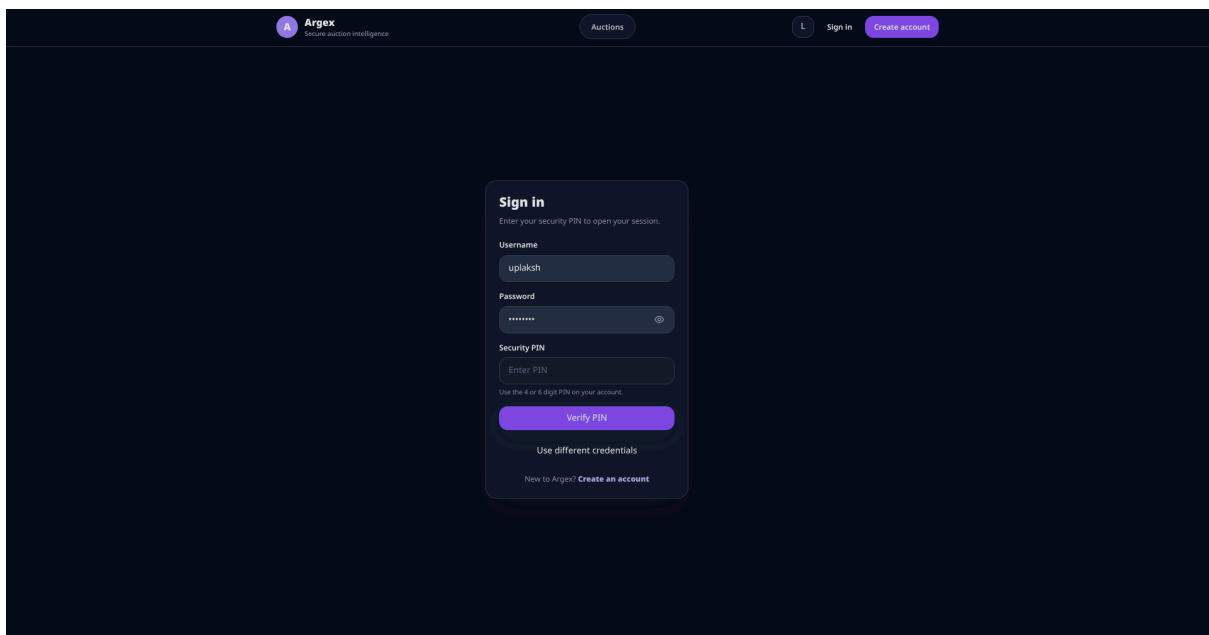




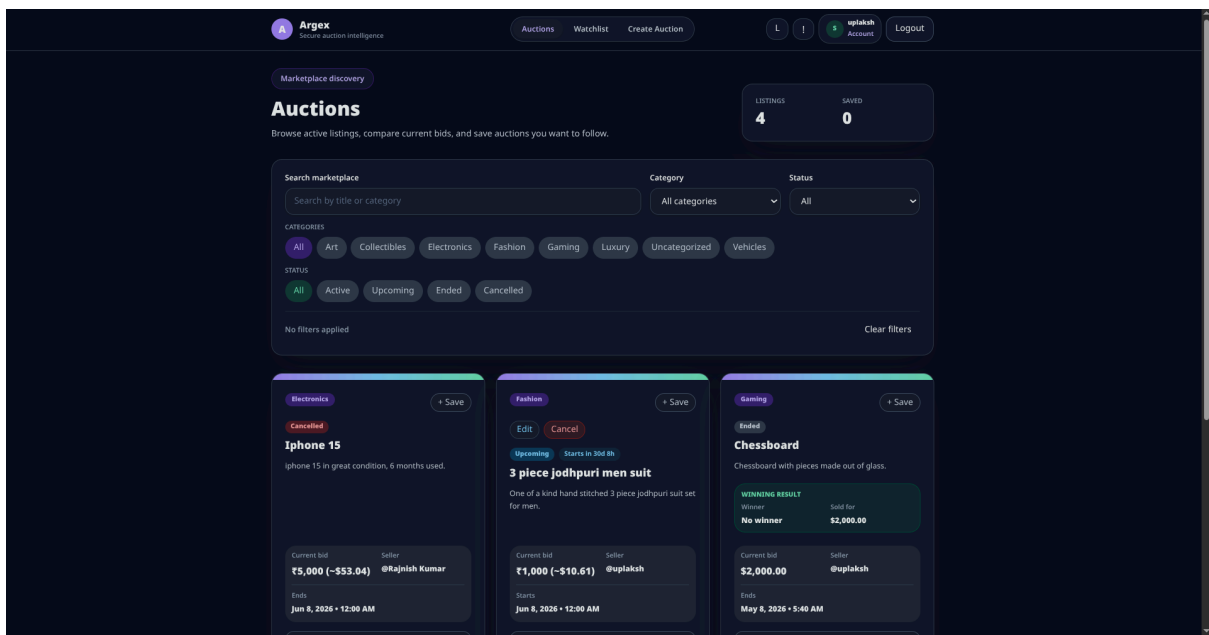
create account



login



auctions



create auctions

Argex

Secure auction intelligence

Auctions

Watchlist

Create Auction

L

!

\$ uplaksh Account

Logout

Seller workspace

Create auction

Launch a verified marketplace listing with clear timing, category, and opening bid.

Title

Collector camera kit

Description

Describe condition, included items, provenance, or delivery details.

Category

Select category

Currency

\$ USD

Starting bid (USD)

500

Start time

DATE

dd/mm/yyyy

HOUR

Hour

MINUTE

00

AM/PM

AM

PM

Leave all start fields blank to start immediately.

End time

DATE

dd/mm/yyyy

HOUR

Hour

MINUTE

00

AM/PM

AM

PM

Cancel

Create Auction

user profile

Argex

Secure auction intelligence

Auctions

Watchlist

Create Auction

L

!

\$ uplaksh Account

Logout

ACCOUNT

uplaksh

uptrouplaksh@gmail.com

Verified Account

Account overview

ROLE

seller

BALANCE

\$10,000.00

PREFERRED CURRENCY

USD

Rates updated 1 minute ago

EMAIL VERIFICATION

Verified

Role actions

Create Auction

Seller Dashboard

Watchlist

Marketplace stats

AUCTIONS CREATED

2

WATCHED

2

BIDS PLACED

1

AUCTIONS WON

0

Security

Security PIN

PIN is enabled for faster protected sign-in.

Current PIN

New PIN

Email PIN Code

Change PIN

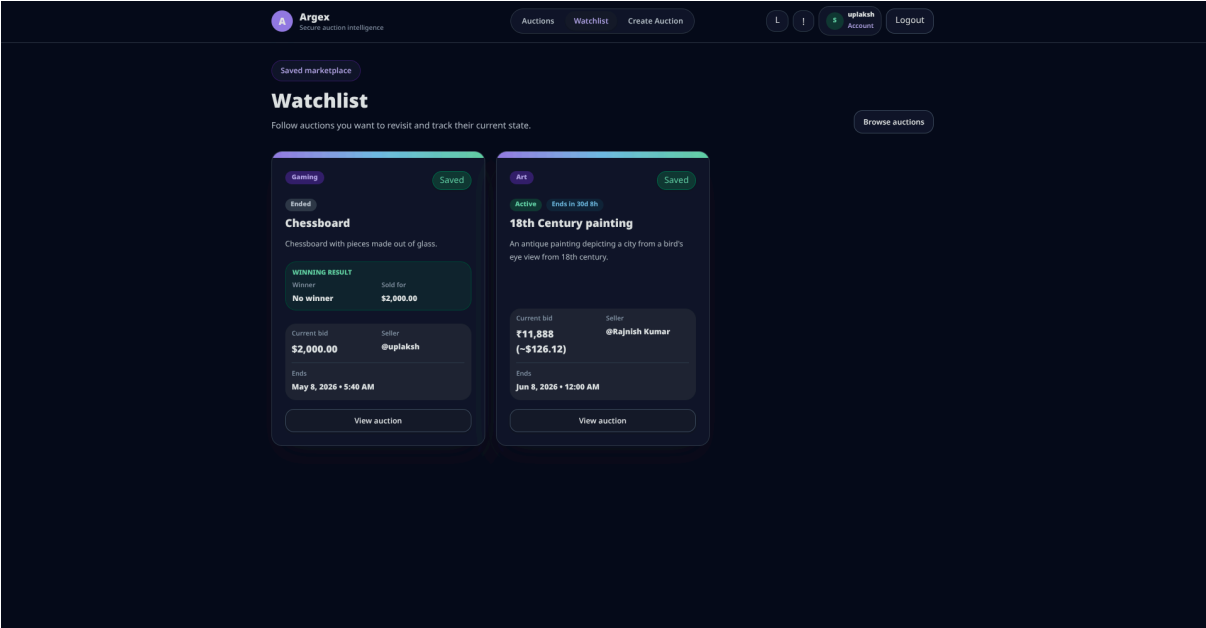
Reset PIN

Enabled

Activity

Recent bidding and watched-auction activity is summarized in marketplace stats. Detailed activity remains available in auction rooms and watchlists.

51



CHAPTER 4: CODING Sample code

File referred:

backend/app/api/routes/bid.py

```
1  from fastapi import APIRouter, Depends
2  from sqlalchemy.orm import Session
3
4  from backend.app.db.session import get_db
5  from backend.app.dependencies.rbac import require_role
6  from backend.app.models.user import User
7
8  from backend.app.schemas.bid import (
9      AutoBidRequest,
10     AutoBidResponse,
11     BidHistoryResponse,
12     BidRequest,
13     BidResponse,
14     BidStatsResponse,
15 )
16 from backend.app.services.bid_service import (
17     create_or_update_auto_bid,
18     disable_user_auto_bid,
19     get_bid_history,
20     get_user_bid_stats,
21     get_user_auto_bid,
22     place_bid as place_bid_service,
23     user_auto_bid_enabled,
24 )
25
26 <?/bids
27 router = APIRouter(prefix="/bids", tags=["Bids"])
28
29 <?/bids/auto
30 @router.post(path: "/auto", response_model=AutoBidResponse) #uptouplaksh *
31 async def upsert_auto_bid(
32     data: AutoBidRequest,
33     db: Session = Depends(get_db),
34     current_user: User = Depends(require_role(["bidder", "seller"])),
35 ):
36     return await create_or_update_auto_bid(db, data, current_user)
```

```

35     return await create_or_update_auto_bid(db, data, current_user)
36
37
38 <@/bids/auto/{auction_id}
39 @router.get( path: "/auto/{auction_id}", response_model=AutoBidResponse)  #uptouplaksh *
40 def get_auto_bid(
41     auction_id: int,
42     db: Session = Depends(get_db),
43     current_user: User = Depends(require_role(["bidder", "seller"])),
44 ):
45     return get_user_auto_bid(db, auction_id, current_user)
46
47 <@/bids/auto/{auction_id}
48 @router.delete( path: "/auto/{auction_id}", response_model=AutoBidResponse) new *
49 def disable_auto_bid(
50     auction_id: int,
51     db: Session = Depends(get_db),
52     current_user: User = Depends(require_role(["bidder", "seller"])),
53 ):
54     return disable_user_auto_bid(db, auction_id, current_user)
55
56 <@/bids/{auction_id}
57 @router.post( path: "/{auction_id}", response_model=BidResponse)  #uptouplaksh *
58 async def place_bid(
59     auction_id: int,
60     bid: BidRequest,
61     db: Session = Depends(get_db),
62     current_user: User = Depends(require_role(["bidder", "seller"])),
63 ):
64     amount = bid.amount
65     bid_record = await place_bid_service(db, auction_id, amount, current_user)
66
67     return BidResponse(
68         auction_id=auction_id,
69         new_price=bid_record.amount,
70         bidder_id=bid_record.bidder_id,
71         bidder_username=bid_record.bidder.username if bid_record.bidder else None,
72         created_at=bid_record.created_at,
73         is_auto=bid_record.is_auto,
74         user_auto_bid_enabled=user_auto_bid_enabled(db, auction_id, current_user),
75     )
76
77 <@/bids/me/stats
78 @router.get( path: "/me/stats", response_model=BidStatsResponse) new *
79 def my_bid_stats(
80     db: Session = Depends(get_db),
81     current_user: User = Depends(require_role(["bidder", "seller", "defender", "admin"])),
82 ):
83     return get_user_bid_stats(db, current_user)
84
85 <@/bids/{auction_id}/history
86 @router.get( path: "/{auction_id}/history", response_model=list[BidHistoryResponse])  #uptouplaksh
87 def bid_history(auction_id: int, db: Session = Depends(get_db)):
88     return get_bid_history(db, auction_id)

```

File referenced:

backend/app/services/security_service.py

```
1  from datetime import datetime, timedelta, timezone
2  from typing import Any
3
4  from fastapi import HTTPException
5  from sqlalchemy.orm import Session
6
7  from backend.app.models.bid import Bid
8  from backend.app.models.security_incident import (
9      IncidentSeverity,
10     IncidentStatus,
11     SecurityIncident,
12 )
13 from backend.app.models.user import User
14 from backend.app.services import logging_service, notification_service
15
16 INCIDENT_THRESHOLD = 50
17 HIGH_SEVERITY_THRESHOLD = 71
18 RECENT_BID_WINDOW_SECONDS = 10
19 REPEATED_BID_WINDOW_SECONDS = 60
20 INCIDENT_DEDUP_WINDOW_SECONDS = 60
21 ABNORMAL_JUMP_MIN_AMOUNT = 25
22 ABNORMAL_JUMP_RATIO = 0.5
23 LAST_SECOND_WINDOW_SECONDS = 120
24 AUTO_LOOP_MEDIUM_ITERATIONS = 3
25 AUTO_LOOP_HIGH_ITERATIONS = 5
26
27
28 def utc_now() → datetime: 1 usage 100% uptouplaksh
29     return datetime.now(timezone.utc)
30
31
32 def ensure_aware_utc(value: datetime) → datetime: 1 usage 100% uptouplaksh
33     if value.tzinfo is None:
34         return value.replace(tzinfo=timezone.utc)
35     return value.astimezone(timezone.utc)
36
37
38 def classify_severity(risk_score: float) → IncidentSeverity: 1 usage 100% uptouplaksh
39
40 def classify_severity(risk_score: float) → IncidentSeverity: 1 usage 100% uptouplaksh
41     if risk_score ≤ 30:
42         return IncidentSeverity.Low
43     if risk_score ≤ 70:
44         return IncidentSeverity.Medium
45     return IncidentSeverity.High
46
47
48 def calculate_risk_score(User: User, action_context: dict[str, Any]) → float: 1 usage 100% uptouplaksh
49     score = 0.0
50
51     if action_context.get(key="rapid_bid_count", default=0) ≥ 5:
52         score += 55
53     elif action_context.get(key="rapid_bid_count", default=0) ≥ 3:
54         score += 20
55
56     if action_context.get(key="repeated_auction_bid_count", default=0) ≥ 3:
57         score += 15
58
59     bid_amount = action_context.get(key="bid_amount", default=0)
60     previous_price = action_context.get(key="previous_price", default=0)
61     bid_jump = max(0, bid_amount - previous_price)
62     abnormal_jump_threshold = max(
63         ABNORMAL_JUMP_MIN_AMOUNT,
64         previous_price * ABNORMAL_JUMP_RATIO,
65     )
66
67     if previous_price > 0 and bid_jump ≥ abnormal_jump_threshold:
68         score += 30
69
70     seconds_to_end = action_context.get("seconds_to_end")
71     if seconds_to_end is not None and 0 ≤ seconds_to_end ≤ LAST_SECOND_WINDOW_SECONDS:
72         score += 20
73
74     auto_loop_iteration = action_context.get(key="auto_loop_iteration", default=0)
75     if auto_loop_iteration ≥ AUTO_LOOP_HIGH_ITERATIONS:
76         score += 55
77     elif auto_loop_iteration ≥ AUTO_LOOP_MEDIUM_ITERATIONS:
```

```

46 def calculate_risk_score(user: User, action_context: dict[str, Any]) → float: 1 usage 1 uptonlaksh
74     elif auto_loop_iteration ≥ AUTO_LOOP_MEDIUM_ITERATIONS:
75         score += 20
76
77     if action_context.get("is_auto") and action_context.get( key: "auto_competitor_count", default: 0) ≥ 2:
78         score += 10
79
80     return min(score, 100.0)
81
82
83 def incident_type_for_context(action_context: dict[str, Any]) → str: 1 usage 1 uptonlaksh
84     if action_context.get( key: "auto_loop_iteration", default: 0) ≥ AUTO_LOOP_MEDIUM_ITERATIONS:
85         return "suspicious_auto_bid_loop"
86     if action_context.get( key: "rapid_bid_count", default: 0) ≥ 5:
87         return "rate_limit_bid_frequency"
88     if action_context.get( key: "rapid_bid_count", default: 0) ≥ 3:
89         return "rapid_bidding_pattern"
90
91     bid_amount = action_context.get( key: "bid_amount", default: 0)
92     previous_price = action_context.get( key: "previous_price", default: 0)
93     bid_jump = max(0, bid_amount - previous_price)
94     abnormal_jump_threshold = max(
95         ABNORMAL_JUMP_MIN_AMOUNT,
96         previous_price * ABNORMAL_JUMP_RATIO,
97     )
98     if previous_price > 0 and bid_jump ≥ abnormal_jump_threshold:
99         return "abnormal_bid_jump"
100
101     seconds_to_end = action_context.get("seconds_to_end")
102     if seconds_to_end is not None and 0 ≤ seconds_to_end ≤ LAST_SECOND_WINDOW_SECONDS:
103         return "last_second_bidding"
104
105     if action_context.get( key: "repeated_auction_bid_count", default: 0) ≥ 3:
106         return "repeated_outbidding"
107
108     return "bid_risk"
109
110

```

```

109
110
111 def build_incident_description(action_context: dict[str, Any]) → str: 1 usage 1 uptonlaksh
112     parts = [
113         f"bid_amount={action_context.get('bid_amount')}",
114         f"previous_price={action_context.get('previous_price')}",
115         f"rapid_bid_count={action_context.get('rapid_bid_count')}",
116         f"repeated_auction_bid_count={action_context.get('repeated_auction_bid_count')}",
117         f"is_auto={action_context.get('is_auto')}",
118     ]
119
120     if action_context.get("seconds_to_end") is not None:
121         parts.append(f"seconds_to_end={round(action_context['seconds_to_end'], 2)}")
122     if action_context.get("auto_loop_iteration"):
123         parts.append(f"auto_loop_iteration={action_context['auto_loop_iteration']}")
124     if action_context.get("auto_competitor_count") is not None:
125         parts.append(f"auto_competitor_count={action_context['auto_competitor_count']}")
126
127     return "Suspicious bidding activity detected: " + " ".join(parts)
128
129
130 def has_recent_duplicate_incident( 1 usage 1 uptonlaksh
131     db: Session,
132     user_id: int,
133     auction_id: int | None,
134     incident_type: str,
135     now: datetime,
136 ) → bool:
137     cutoff = now - timedelta(seconds=INCIDENT_DEDUP_WINDOW_SECONDS)
138     return (
139         db.query(SecurityIncident.id)
140         .filter(
141             *criteria: SecurityIncident.user_id == user_id,
142             SecurityIncident.auction_id == auction_id,
143             SecurityIncident.incident_type == incident_type,
144             SecurityIncident.status == IncidentStatus.open,
145             SecurityIncident.created_at ≥ cutoff,
146         )
147         .count() > 0
148     )

```

```

130 def has_recent_duplicate_incident( 1 usage ⚠uptouplaksh
146     )
147     .first()
148     is not None
149 )
150
151
152 def create_incident( 1 usage ⚠uptouplaksh
153     db: Session,
154     user_id: int,
155     auction_id: int | None,
156     incident_type: str,
157     risk_score: float,
158     description: str,
159     created_at: datetime,
160 ) → SecurityIncident | None:
161     if has_recent_duplicate_incident(db, user_id, auction_id, incident_type, created_at):
162         return None
163
164     severity = classify_severity(risk_score)
165     incident = SecurityIncident(
166         user_id=user_id,
167         auction_id=auction_id,
168         incident_type=incident_type,
169         severity=severity,
170         risk_score=risk_score,
171         description=description,
172         created_at=created_at,
173         status=IncidentStatus.open,
174     )
175     db.add(incident)
176     db.flush()
177
178     logging_service.log_action(
179         db=db,
180         user_id=user_id,
181         action_type="CREATE_SECURITY_INCIDENT",
182         entity_type="SECURITY_INCIDENT"
183     )
184
185     details={
186         "risk_score": risk_score,
187         "severity": severity.value,
188         "incident_type": incident_type,
189         "auction_id": auction_id,
190     },
191 )
192
193 if severity == IncidentSeverity.high:
194     notification_service.create_notification(
195         db=db,
196         user_id=user_id,
197         type="HIGH_SEVERITY_SECURITY_INCIDENT",
198         message=f"A high-severity security incident of type '{incident_type}' was detected on your account. Please
199     )
200     print(
201         "[SECURITY ALERT] "
202         f"user={user_id} auction={auction_id} type={incident_type} score={risk_score}"
203     )
204
205     return incident
206
207 def evaluate_bid_risk( 3+ usages ⚠uptouplaksh
208     db: Session,
209     user: User,
210     auction_id: int,
211     bid_amount: float,
212     previous_price: float,
213     auction_end_time: datetime,
214     bid_time: datetime,
215     is_auto: bool,
216     auto_loop_iteration: int = 0,
217     auto_competitor_count: int = 0,
218 ) → float:

```

```

207 def evaluate_bid_risk( 3+ usages ⚠uptouplaksh
218 ) → float:
219     rapid_cutoff = bid_time - timedelta(seconds=RECENT_BID_WINDOW_SECONDS)
220     repeated_cutoff = bid_time - timedelta(seconds=REPEATED_BID_WINDOW_SECONDS)
221
222     rapid_bid_count = (
223         db.query(Bid.id)
224         .filter(
225             *criterion: Bid.bidder_id == user.id,
226             Bid.created_at ≥ rapid_cutoff,
227         )
228         .count()
229     )
230     repeated_auction_bid_count = (
231         db.query(Bid.id)
232         .filter(
233             *criterion: Bid.bidder_id == user.id,
234             Bid.auction_id == auction_id,
235             Bid.created_at ≥ repeated_cutoff,
236         )
237         .count()
238     )
239
240     end_time = ensure_aware_utc(auction_end_time)
241     seconds_to_end = (end_time - bid_time).total_seconds()
242     action_context = {
243         "auction_id": auction_id,
244         "bid_amount": bid_amount,
245         "previous_price": previous_price,
246         "rapid_bid_count": rapid_bid_count,
247         "repeated_auction_bid_count": repeated_auction_bid_count,
248         "seconds_to_end": seconds_to_end,
249         "is_auto": is_auto,
250         "auto_loop_iteration": auto_loop_iteration,
251         "auto_competitor_count": auto_competitor_count,
252     }
253
254     risk_score = calculate_risk_score(user, action_context)
255
256     user.cumulative_risk_score = (user.cumulative_risk_score or 0.0) + risk_score
257     if not user.is_suspected and user.cumulative_risk_score ≥ HIGH_SEVERITY_THRESHOLD:
258         user.is_suspected = True
259
260     if risk_score ≥ INCIDENT_THRESHOLD:
261         create_incident(
262             db,
263             user_id=user.id,
264             auction_id=auction_id,
265             incident_type=incident_type_for_context(action_context),
266             risk_score=risk_score,
267             description=build_incident_description(action_context),
268             created_at=bid_time,
269         )
270
271     return risk_score
272
273
274 def list_incidents( 2+ usages ⚠uptouplaksh
275     db: Session,
276     status: IncidentStatus | None = None,
277     severity: IncidentSeverity | None = None,
278 ) → List[SecurityIncident]:
279     query = db.query(SecurityIncident)
280     if status is not None:
281         query = query.filter(SecurityIncident.status == status)
282     if severity is not None:
283         query = query.filter(SecurityIncident.severity == severity)
284     return query.order_by(SecurityIncident.created_at.desc()).all()
285
286
287 def resolve_incident(db: Session, incident_id: int) → SecurityIncident: 2+ usages ⚠uptouplaksh
288     incident = db.query(SecurityIncident).filter(SecurityIncident.id == incident_id).first()
289     if not incident:
290         raise HTTPException(status_code=404, detail="Security incident not found")

```

File referenced:

frontend/src/hooks/useAuctionWebSocket.js

```
1  import { useEffect, useRef, useState } from 'react'
2
3  const WS_BASE_URL = import.meta.env.VITE_WS_BASE_URL || 'ws://localhost:8000'
4  const RECONNECT_DELAYS : number[] = [1000, 2000, 4000, 8000]
5
6  function parseSocketMessage(rawMessage) : ... { Show usages
7    try {
8      return JSON.parse(rawMessage)
9    } catch {
10     return null
11    }
12  }
13
14  function useAuctionWebSocket(auctionId, onEvent) : string { Show usages
15    const [status : string , setStatus] = useState( initialState: auctionId ? 'connecting' : 'idle')
16    const socketRef : RefObject<null> = useRef( initialValue: null)
17    const reconnectTimerRef : RefObject<null> = useRef( initialValue: null)
18    const reconnectAttemptRef : RefObject<number> = useRef( initialValue: 0)
19    const manuallyClosedRef : RefObject<boolean> = useRef( initialValue: false)
20    const onEventRef : RefObject<unknown> = useRef(onEvent)
21
22    useEffect( effect: () : void => {
23      onEventRef.current = onEvent
24    }, deps: [onEvent])
25
26    useEffect( effect: () : function(): void => {
27      if (!auctionId) {
28        const idleTimer : number = setTimeout( handler: () : void => setStatus( value: 'idle'), timeout: 0)
29        return () : void => clearTimeout(idleTimer)
30      }
31
32      manuallyClosedRef.current = false
33
34      function clearReconnectTimer() : void { Show usages
35        if (reconnectTimerRef.current) {
36          clearTimeout(reconnectTimerRef.current)
37          reconnectTimerRef.current = null
38        }
39      }
40    })
41  }
```

```

14 function useAuctionWebSocket(auctionId, onEvent) : string { Show usages
26   useEffect( effect: () : function(): void => {
39     }
40
41     function connect() : void { Show usages
42       clearReconnectTimer()
43       setStatus( value: 'connecting')
44
45       const socket : WebSocket = new WebSocket( url: `${WS_BASE_URL}/ws/auctions/${auctionId}`)
46       socketRef.current = socket
47
48       socket.onopen = () : void => {
49         reconnectAttemptRef.current = 0
50         setStatus( value: 'connected')
51       }
52
53       socket.onmessage = (event : MessageEvent) : void => {
54         const message = parseSocketMessage(event.data)
55
56         if (!message) {
57           return
58         }
59
60         if (message.type === 'ping') {
61           socket.send(JSON.stringify( value: { type: 'pong' } ))
62           return
63         }
64
65         onEventRef.current?.(message)
66       }
67
68       socket.onerror = () : void => {
69         setStatus( value: 'error')
70       }
71
72       socket.onclose = () : void => {
73         if (manuallyClosedRef.current) {
74           setStatus( value: 'disconnected')
75         }
76       }
77
78       setStatus( value: 'reconnecting')
79       const delay : number = RECONNECT_DELAYS[Math.min(reconnectAttemptRef.current, RECONNECT_DELAYS.length - 1)]
80       reconnectAttemptRef.current += 1
81       reconnectTimerRef.current = setTimeout(connect, delay)
82     }
83   }
84
85   const connectTimer : number = setTimeout(connect, timeout: 0)
86
87   return () : void => {
88     manuallyClosedRef.current = true
89     clearTimeout(connectTimer)
90     clearReconnectTimer()
91
92     if (socketRef.current) {
93       socketRef.current.close()
94       socketRef.current = null
95     }
96   }
97   }, deps: [auctionId])
98
99   return status
100 }
101
102 export default useAuctionWebSocket Show usages
103

```

CHAPTER 5: LIMITATIONS OF SYSTEM

- **Internet Dependency:**
ARGEX requires a stable internet connection for real-time bidding, WebSocket communication, and live notification updates.
- **Limited Payment Integration:**
The current system uses an internal wallet and balance simulation model and does not include integration with real online payment gateways or banking systems.
- **Basic Security Monitoring Scope:**
Although the system includes anomaly detection and risk scoring, it does not implement advanced AI-based threat intelligence or enterprise-level cybersecurity analytics.
- **Scalability Constraints:**
The current deployment is designed for academic and small-scale operational environments and may require infrastructure optimization for handling large-scale concurrent users.
- **Email-Based Verification Dependency:**
User verification and recovery functionalities currently depend on SMTP email services, which may be affected by third-party email delivery limitations.
- **Limited Mobile Optimization:**
While the platform is responsive, a dedicated mobile application for Android or iOS is not implemented in the current version of the system.

CHAPTER 6: PROPOSED ENHANCEMENTS

- AI-Based Threat Detection
- Payment Gateway Integration
- Mobile Application Development
- Advanced Analytics Dashboard
- Multi-Language Support
- Cloud Deployment and Scalability
- Biometric and Multi-Factor Authentication
- Blockchain-Based Transaction Verification
- Advanced Seller Analytics
- Push Notification Integration

CHAPTER 7: CONCLUSION

ARGEX successfully demonstrates the development of a secure and real-time online auction platform integrated with cybersecurity monitoring capabilities. The system provides functionalities such as live bidding, auto-bidding, wallet management, role-based access control, real-time notifications, and responsive user interaction. The integration of anomaly detection, risk scoring, and incident monitoring enhances platform security and differentiates the system from conventional auction platforms.

The project combines full-stack web development, real-time communication, and cyber-aware system design within a single scalable platform. ARGEX not only improves the efficiency and transparency of online auctions but also highlights the importance of operational security and intelligent monitoring in modern web-based marketplace systems.

CHAPTER 8: Bibliography

Books

1. *Designing Data-Intensive Applications* – Martin Kleppmann (2017).
2. *Web Application Security* – Andrew Hoffman (2020).
3. *REST API Design Rulebook* – Mark Masse (2011).
4. *Clean Code: A Handbook of Agile Software Craftsmanship* – Robert C. Martin (2008).
5. *The Web Application Hacker's Handbook* – Dafydd Stuttard and Marcus Pinto (2011).

Research Papers & Journals

6. Research Papers on Real-Time Auction Systems and E-Commerce Platforms – IEEE Xplore Digital Library.
7. Research Studies on Behavioral Anomaly Detection in Online Marketplaces – ACM Digital Library.
8. Research Material on Role-Based Access Control (RBAC) Systems in Web Applications.
9. Studies on Fraud Detection and Secure Transaction Systems in E-Commerce Platforms.
10. Research Papers on Real-Time Communication Using WebSocket Architecture.

Official Documentation

11. FastAPI Official Documentation – Modern Python Web Framework Guide.
<https://fastapi.tiangolo.com/>
12. React.js Official Documentation – Frontend Development Library Guide.
<https://react.dev/>
13. PostgreSQL Official Documentation – Relational Database Management System Guide.
<https://www.postgresql.org/docs/>
14. Tailwind CSS Official Documentation – Utility-First CSS Framework Guide.
<https://tailwindcss.com/docs/>
15. Python Official Documentation – Python Programming Language Reference.
<https://docs.python.org/3/>
16. WebSocket API Documentation – Real-Time Communication Standards.
https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API
17. JWT Official Introduction – Secure Token-Based Authentication Concepts.
<https://jwt.io/introduction>

Websites & Technical Resources

18. OWASP Top 10 – Web Application Security Risks and Best Practices.
<https://owasp.org/www-project-top-ten/>
19. NIST Cybersecurity Framework – Security and Risk Management Guidelines.
<https://www.nist.gov/cyberframework>
20. MITRE ATT&CK Framework – Threat Intelligence and Adversarial Behavior Knowledge Base.
<https://attack.mitre.org/>
21. Vite Official Documentation – Frontend Build Tool and Development Server Guide.
<https://vitejs.dev/>

YouTube Channels & Learning Resources

22. Hitesh Choudhary – Full Stack Development and Backend Architecture Tutorials.
23. NetworkChuck – Networking and Cybersecurity Concepts.
24. freeCodeCamp – Open Source Programming and Web Development Tutorials.

Other Sources

25. Linux Manual Pages – System Commands and Server Utilities Reference.
26. SMTP Protocol Documentation for Email Authentication Systems.
27. PostgreSQL Transaction Management and Database Integrity Concepts.
28. Research Material on Event-Driven System Architecture and Real-Time Synchronization.